

Движение данных в MRP системах

Этапы выполнения параллельного запроса

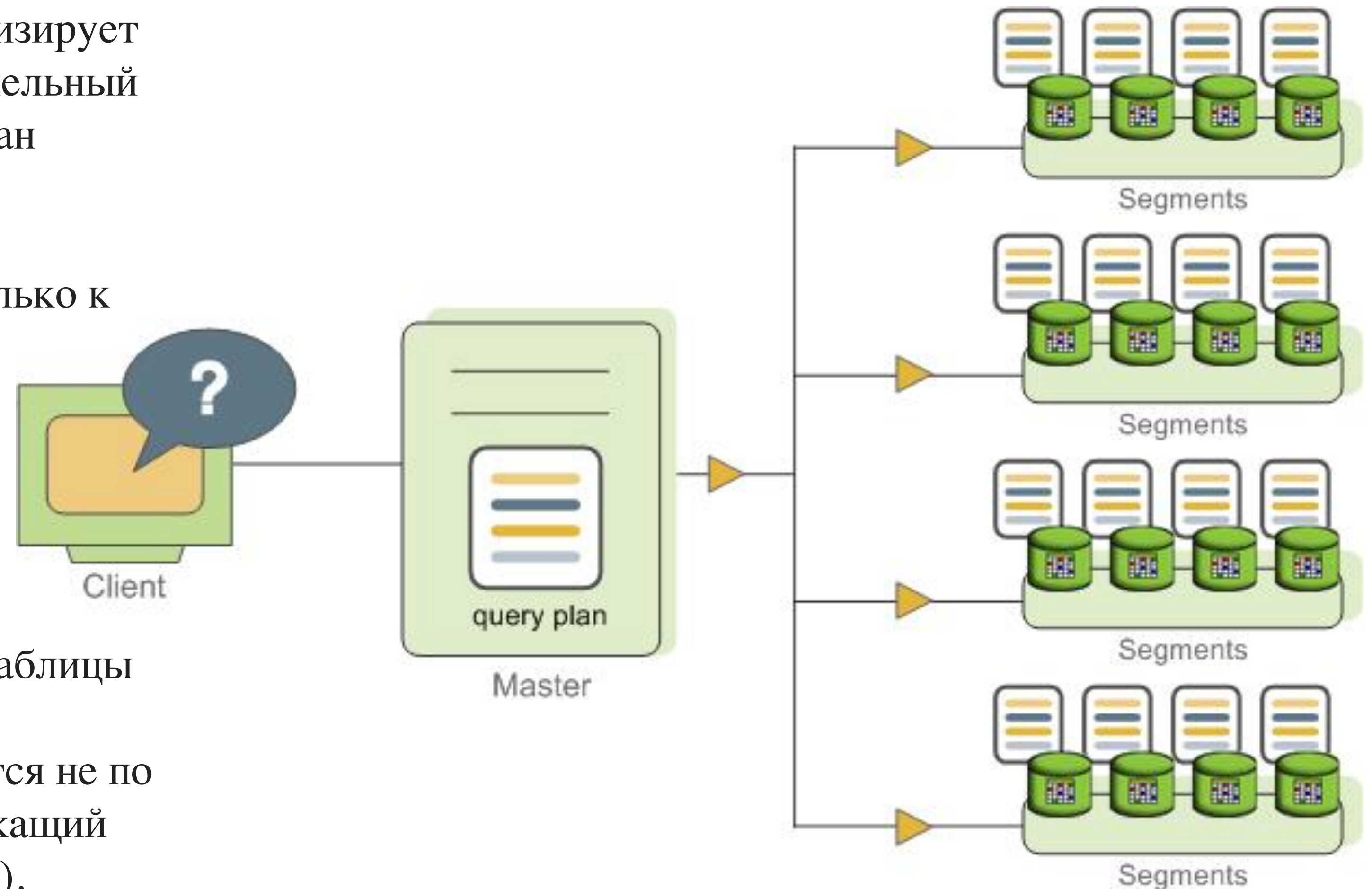
Координатор получает, анализирует и оптимизирует запрос. В результате получается либо параллельный (для нескольких сегментов), либо целевой план запроса (для одного сегмента).

Определенные запросы могут обращаться только к данным в одном сегменте.

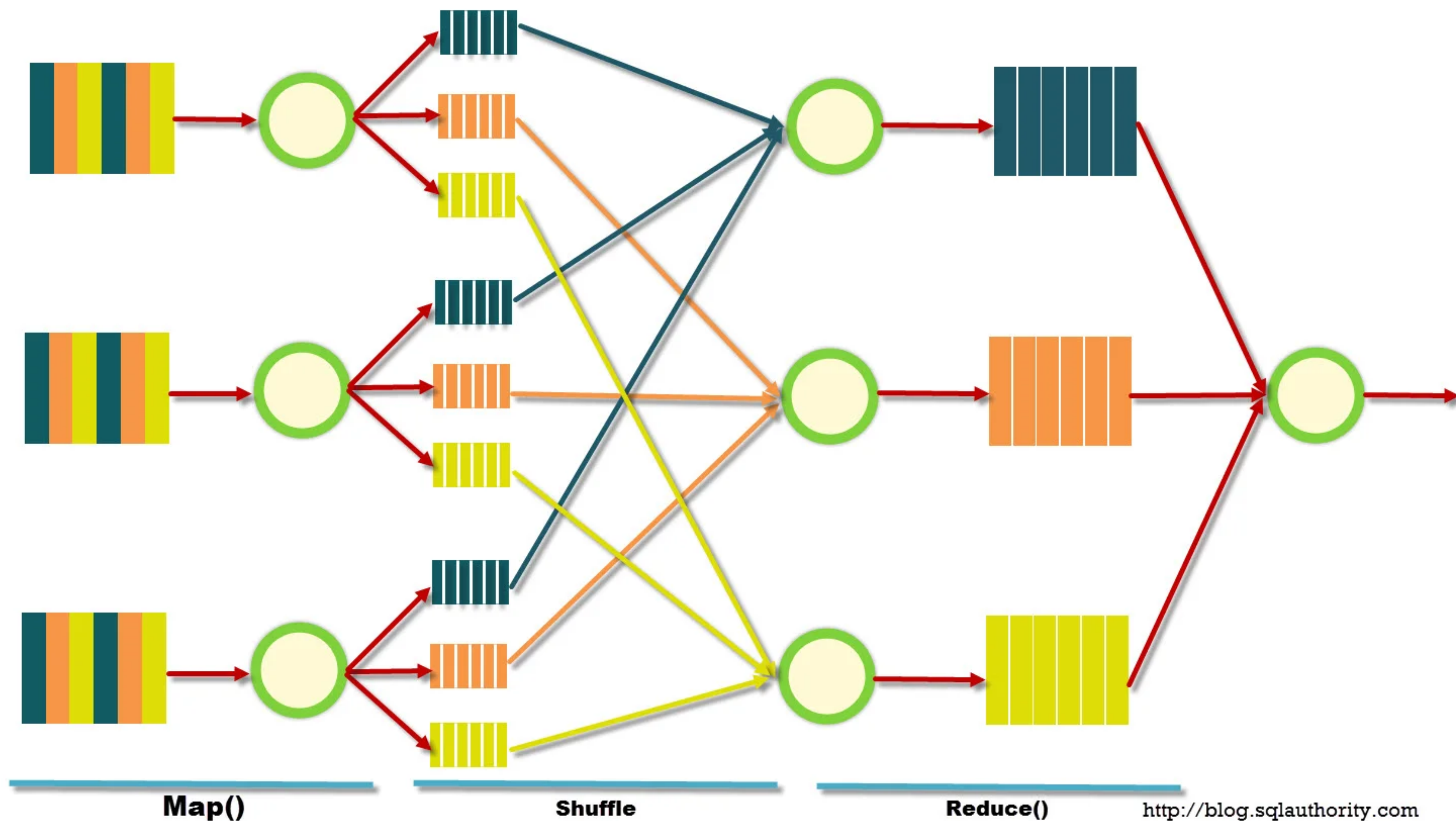
Например:

- INSERT
- UPDATE
- DELETE
- SELECT, который фильтрует по столбцу(столбцам) ключа распределения таблицы

В подобных запросах план запроса рассылается не по всем сегментам, а нацелен на сегмент, содержащий затронутую или релевантную строку (строки).



MapReduce в МРР системах



Какие бывают движения данных

Основная задача распределенных систем - равномерно нагрузить каждый сегмент в MPP, что означает добиться локальности выполнения операций. Так как обрабатываемые данные могут находиться на разных сегментах, то необходимо эффективно перемещать данные из сегмента в сегмент.

Какие операции могут вызывать движение данных:

- Операции агрегации
- Операции соединения (джоины)

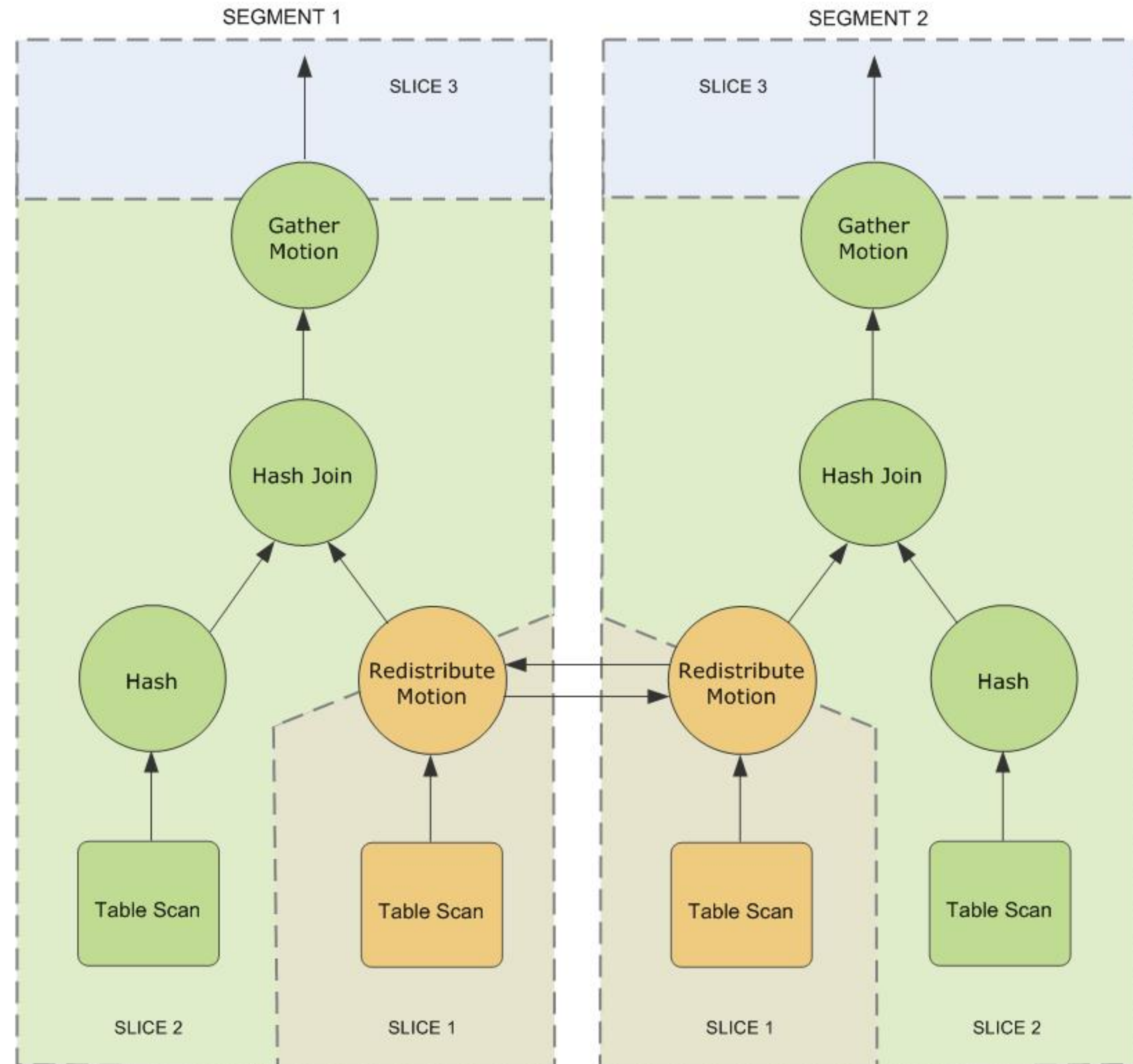
Какие движения данных существуют:

- Redistribute Motion
- Broadcast Motion
- Gather Motion

Redistribute Motion

Если ключ соединения таблиц не соответствует ключу распределения, то возможна ситуация при которых соединяемые данные физически расположены на разных серверах.

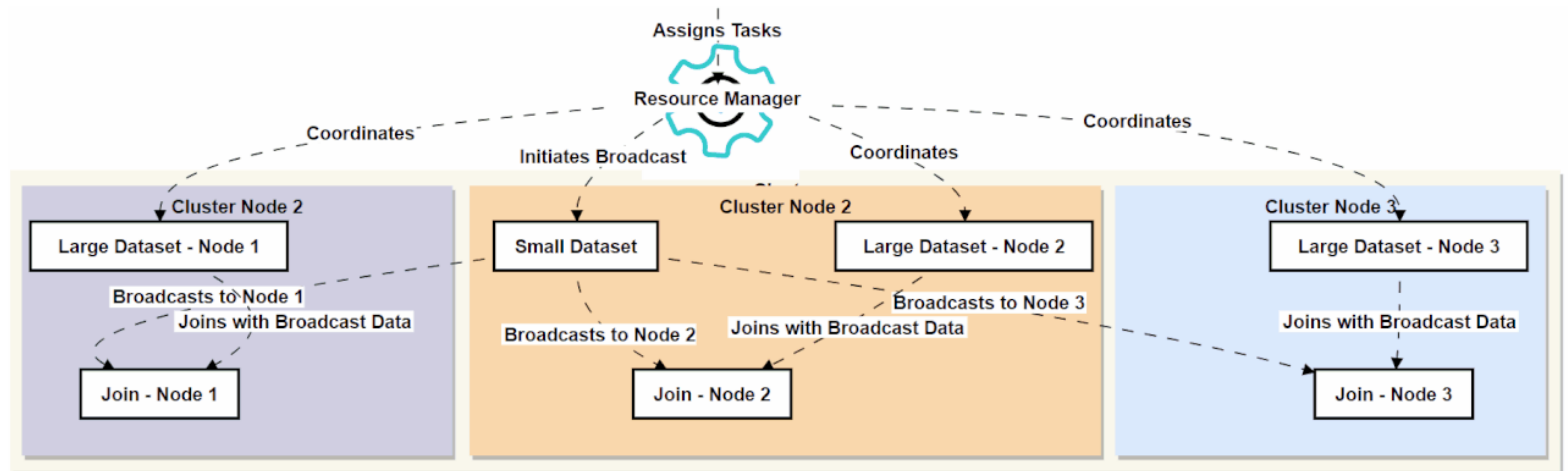
В этом случае для выполнения запроса системе необходимо автоматически перераспределить данные по другому ключу дистрибьюции.



Broadcast Motion

Broadcast Motion выбирает наименьшую из таблиц и дублирует данные на все сегменты.

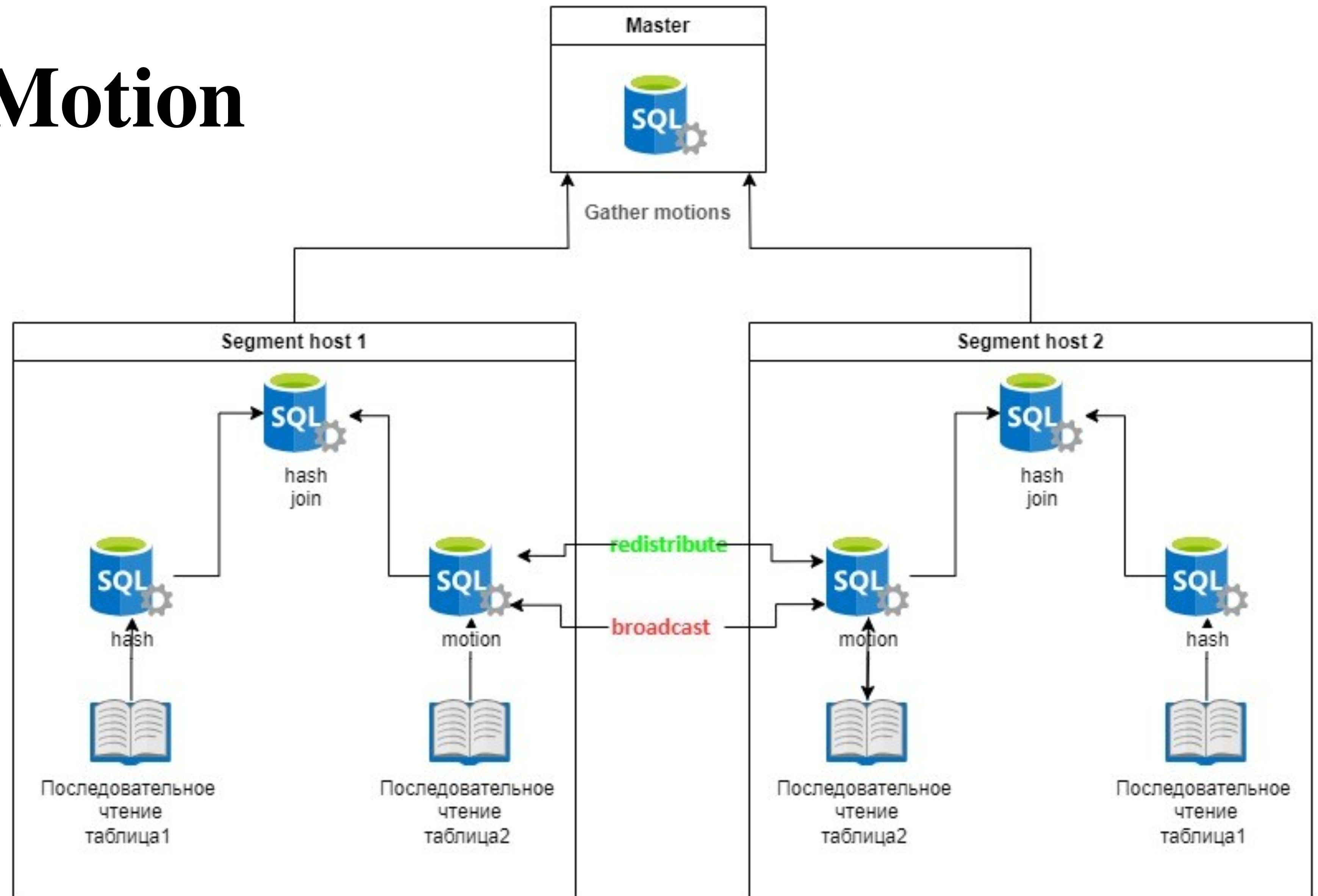
Этот способ может быть не таким оптимальным, как предыдущий, поэтому оптимизатор обычно выбирает его для небольших таблиц.



Gather Motion

Gather Motion -
объединяет на мастере
результаты с сегментов.

Данный шаг
присутствует в каждом
запросе и является
последним шагом плана.



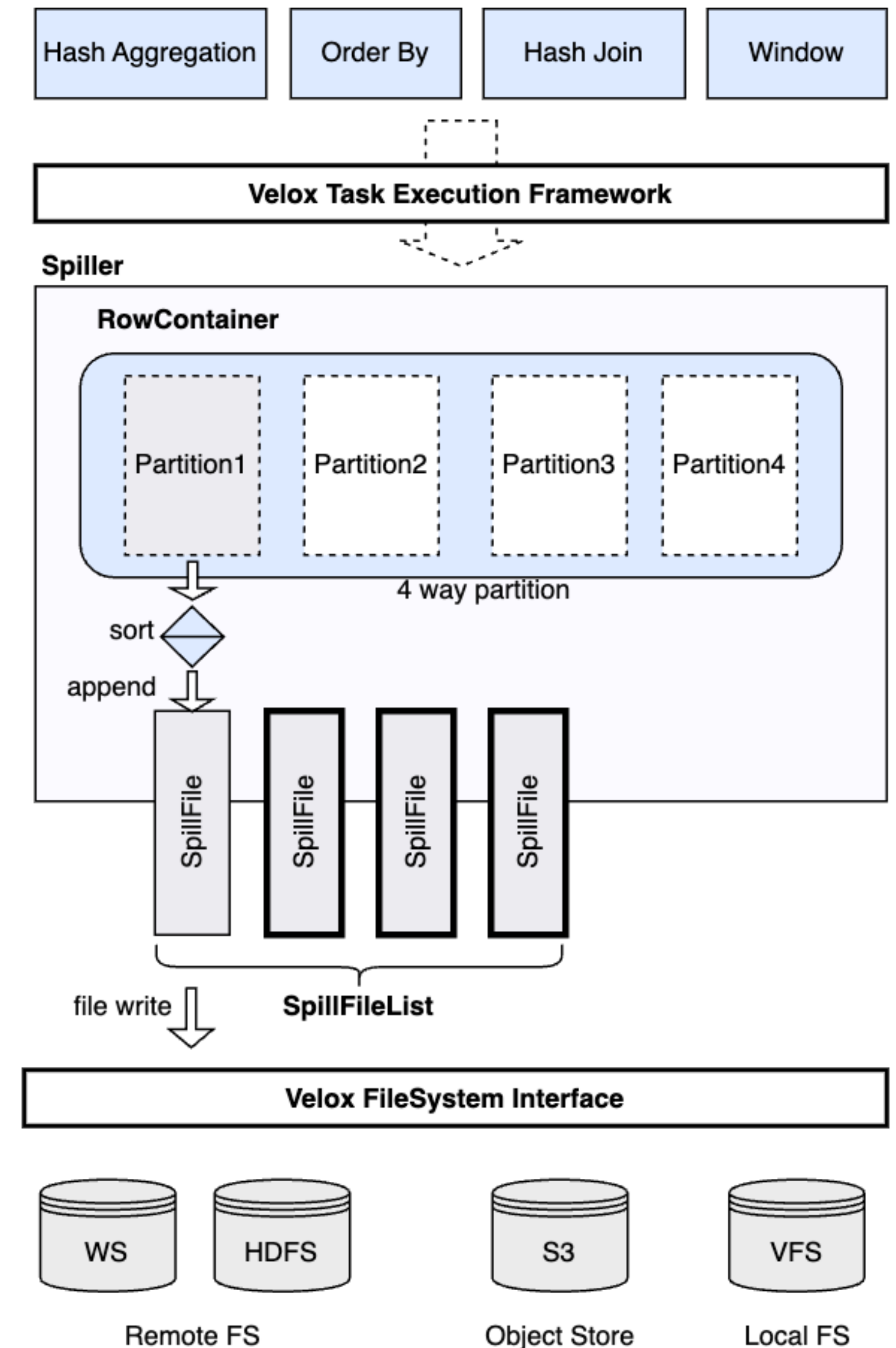
Spill-файлы

Спиллы — это некоторый генерируемый дополнительный объем данных на жестком диске, который используется для выполнения запроса.

Спиллы появляются, когда для запроса нужно хранить данных больше, чем предоставлено оперативной памятью.

Причины генерации Spill-файлов:

- обработка огромного объема данных без фильтров;
- создание хэш-таблицы на основе таблицы большого объема;
- DISTINCT на большом количестве полей;
- использование ORDER BY;
- Использование оконных функций.



**Что делать, чтобы избежать плохих
движений данных и спиллов?**

- 1) **EXPLAIN ANALYSE**
- 2) **Умение читать план запроса**

Пример с распределением не по ключу соединения

Таблица клиентов

```
CREATE TABLE customers
(
  customer_id  INTEGER,
  customer_name VARCHAR(25)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Таблица заказов

```
CREATE TABLE orders
(
  order_id  INTEGER,
  customer_id INTEGER,
  amount    DECIMAL(6, 2)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (order_id);
```

Пример с распределением не по ключу соединения

EXPLAIN

```
SELECT c.customer_id,  
       c.customer_name AS customer_name,  
       SUM(o.amount) AS total_purchases  
FROM customers c  
      JOIN orders o USING (customer_id)  
GROUP BY c.customer_id,  
         c.customer_name  
ORDER BY total_purchases DESC  
LIMIT 5;
```

QUERY PLAN

Limit

-> Gather Motion 4:1 (slice3; segments: 4)

Merge Key: (pg_catalog.sum((sum(o.amount))))

-> Limit

-> Sort

Sort Key: (pg_catalog.sum((sum(o.amount))))

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Redistribute Motion 4:4 (slice2; segments: 4)

Hash Key: c.customer_id, c.customer_name

-> Result

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Hash Join

Hash Cond: (o.customer_id = c.customer_id)

-> Seq Scan on orders o

-> Hash

-> Broadcast Motion 4:4 (slice1; segments: 4)

-> Seq Scan on customers c

Optimizer: Pivotal Optimizer (GPORCA)

(20 rows)

Пример с распределением по ключу соединения

Таблица клиентов

```
CREATE TABLE customers
(
  customer_id  INTEGER,
  customer_name VARCHAR(25)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Таблица заказов

```
CREATE TABLE orders
(
  order_id  INTEGER,
  customer_id INTEGER,
  amount    DECIMAL(6, 2)
)
WITH (appendoptimized = true)
DISTRIBUTED BY (customer_id);
```

Пример с распределением по ключу соединения

```
EXPLAIN
SELECT c.customer_id,
       c.customer_name AS customer_name,
       SUM(o.amount) AS total_purchases
FROM customers c
      JOIN orders o USING (customer_id)
GROUP BY c.customer_id,
         c.customer_name
ORDER BY total_purchases DESC
LIMIT 5;
```

QUERY PLAN

Limit

-> Gather Motion 4:1 (slice1; segments: 4)

Merge Key: (sum(o.amount))

-> Limit

-> Sort

Sort Key: (sum(o.amount))

-> HashAggregate

Group Key: c.customer_id, c.customer_name

-> Hash Join

Hash Cond: (o.customer_id = c.customer_id)

-> Seq Scan on orders o

-> Hash

-> Seq Scan on customers c

Optimizer: Pivotal Optimizer (GPORCA)

(14 rows)

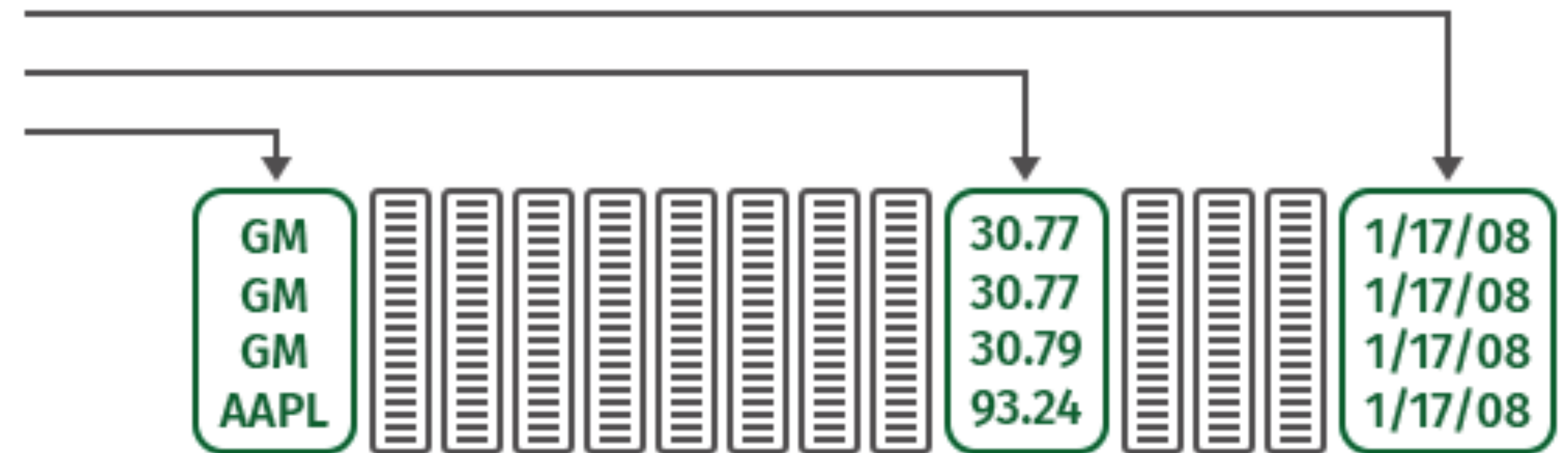
Чем еще отличаются MPP-системы от локальных?

Если мы читаем строки, то,
например, для выполнения команды:

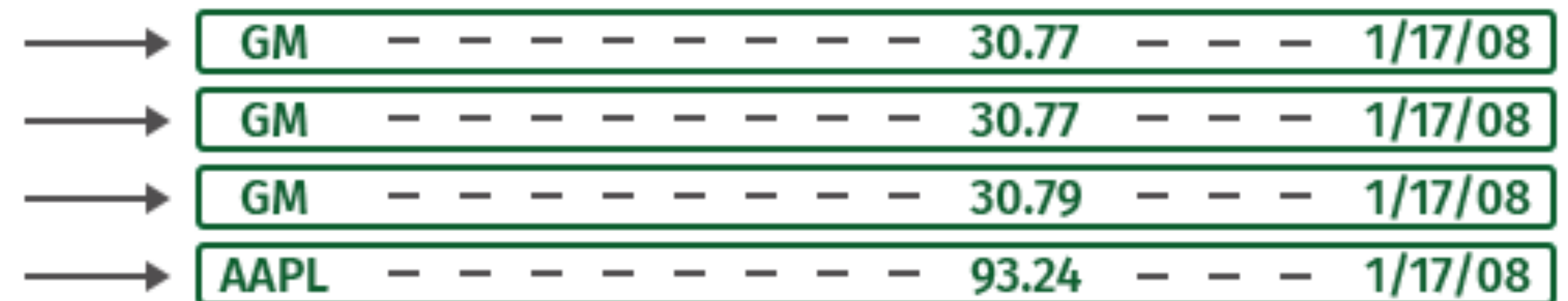
```
SELECT 1,11,15  
FROM table1
```

нам придется читать всю таблицу.
Это огромный объем информации.
В данном случае колоночный подход
выгоднее. Он позволяет считать
только три нужных нам столбца,
экономя память и время

**Хранение
по столбцам**
Чтение трёх столбцов



**Хранение
по строкам**
Чтение всех столбцов



Что такое Parquet

Это бинарный, колонно-ориентированный формат хранения данных, изначально созданный для экосистемы hadoop. Разработчики уверяют, что данный формат хранения идеален для big data (неизменяемых).

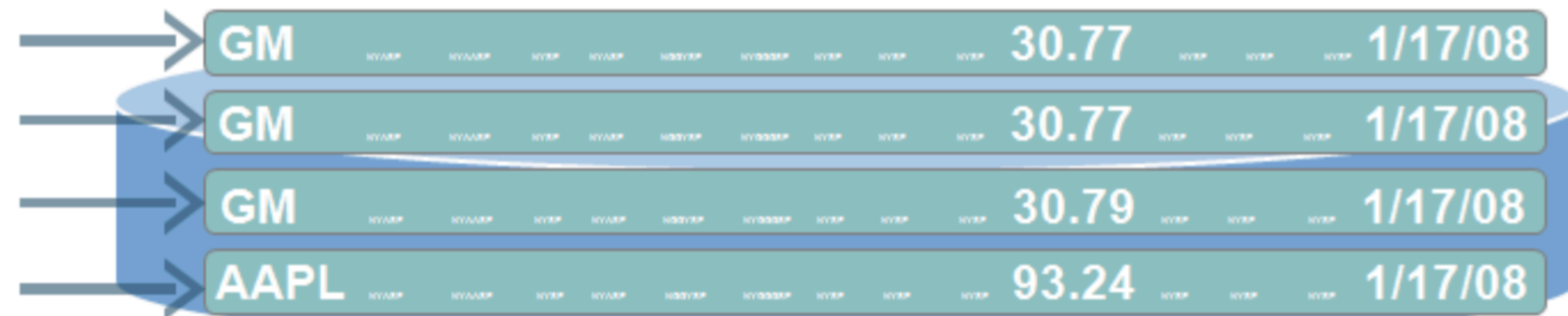
Хранение по столбцам

Чтение 3х столбцов



Хранение по строкам

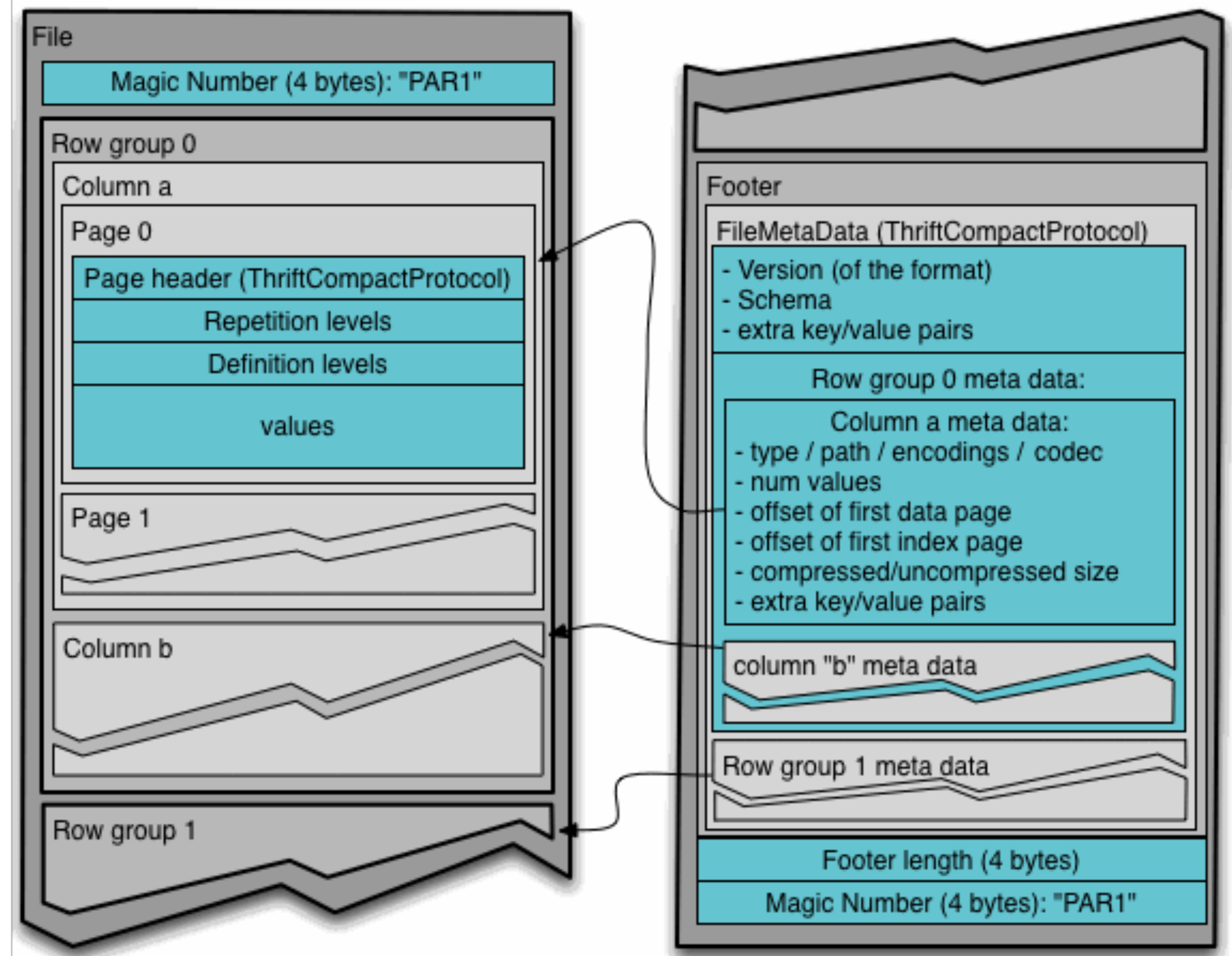
Чтение всех столбцов



Как выглядит структура Parquet файлов

Файлы имеют несколько уровней разбиения на части, благодаря чему возможно довольно эффективное параллельное исполнение операций поверх них:

- Row-group — это разбиение, позволяющее параллельно работать с данными на уровне Map-Reduce
- Column chunk — разбиение на уровне колонок, позволяющее распределять IO операции
- Page — Разбиение колонок на страницы, позволяющее распределять работу по кодированию и сжатию



Поколоночное vs Построчное хранение данных

CSV - пример построчного хранения, Parquet - пример поколоночного хранения данных.
Рассмотрим пример формирования файлов:

A	B	C
A1	B1	C1
A2	B2	C2
A3	B3	C3

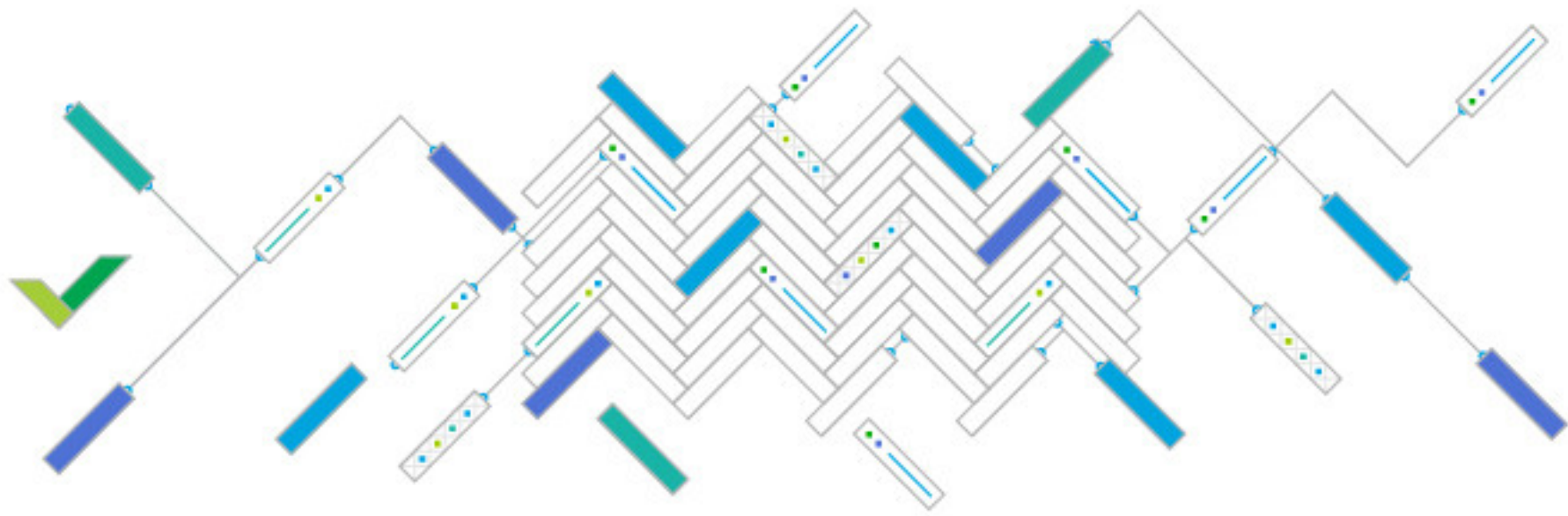
Тогда в текстовом файле, скажем, csv мы бы хранили данные на диске примерно так:

A1	B1	C1	A2	B2	C2	A3	B3	C3
----	----	----	----	----	----	----	----	----

В случае с Parquet:

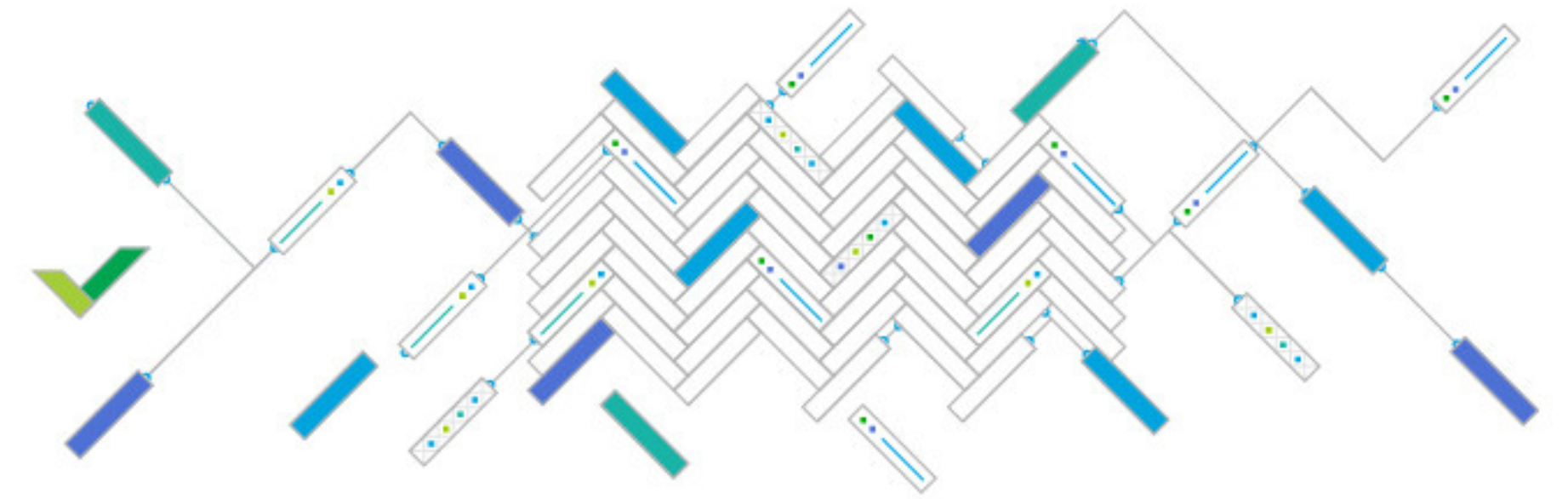
A1	A2	A3	B1	B2	B3	C1	C2	C3
----	----	----	----	----	----	----	----	----

Достоинства и недостатки колодочного хранения



Достоинства:

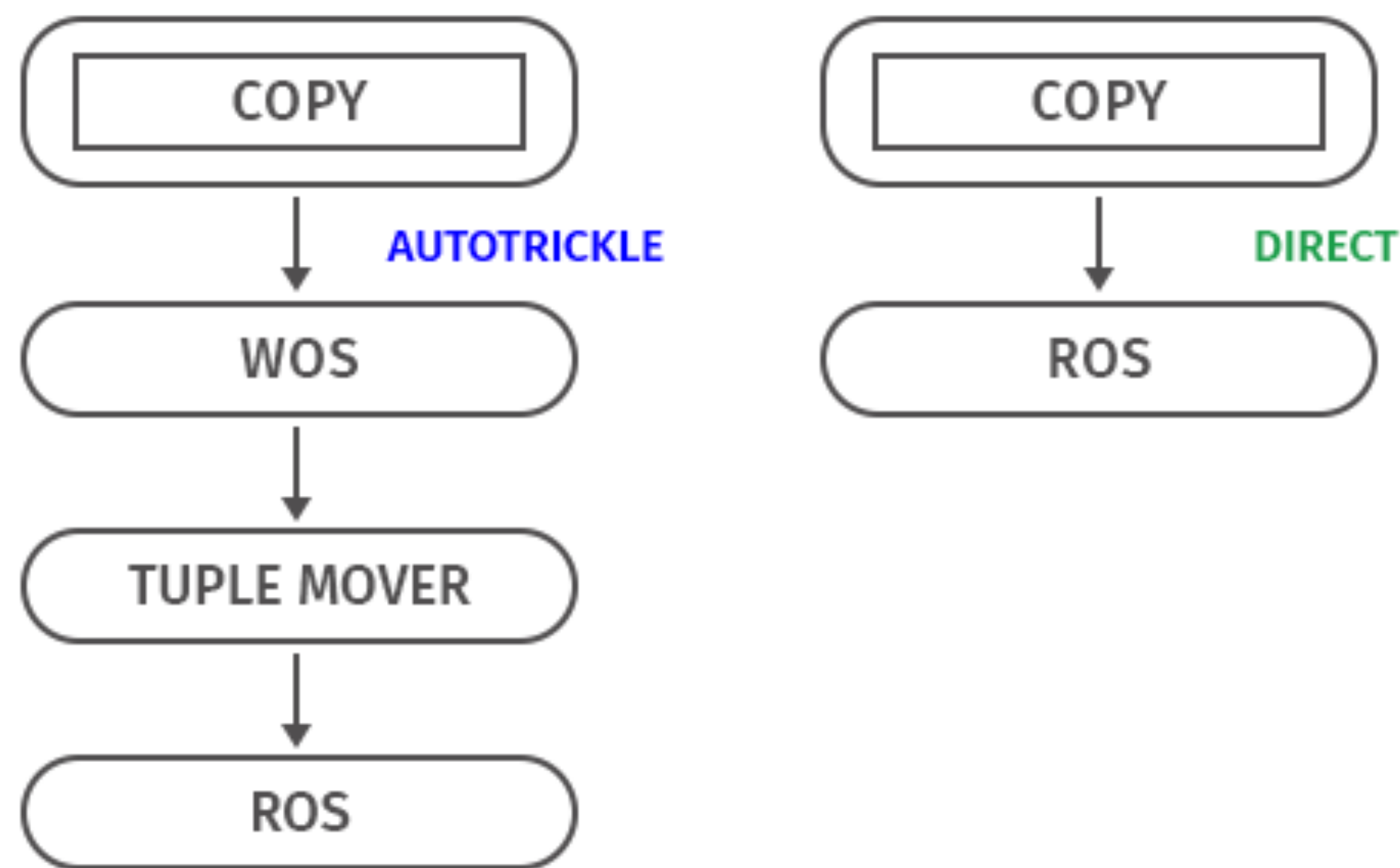
- Несмотря на то, что они и созданы для hdfs, данные могут храниться и в других файловых системах, таких как GlusterFs или поверх NFS
- По сути это просто файлы, а значит с ними легко работать, перемещать, бэкапить и реплицировать.
- Колончатый вид позволяет значительно ускорить работу аналитика, если ему не нужны все колонки сразу.
- Минимизированное хранение с точки зрения занимаемого места.
- Скорость чтения по сравнению с использованием других файловых форматов.



Недостатки:

- Колончатый вид заставляет задумываться о схеме и типах данных.
- Parquet не всегда обладает нативной поддержкой в других продуктах.
- Не поддерживает изменение данных и эволюцию схемы. Чтобы что-то изменить в уже существующем файле, нельзя обойтись без перезаписи, разве что можно добавить новую колонку.
- Не поддерживаются транзакции, так как это обычные файлы а не БД.

С какими проблемами сталкивается колоночное хранение



Вне зависимости от того, куда записаны данные — в WOS или в ROS — они доступны сразу. Но из WOS чтение идет медленнее, потому что данные там не сгруппированы.

В таких системах должен присутствовать инструмент-уборщик, который выполняет две операции:

- Moveout — сжимает и сортирует данные в WOS, перемещает их в ROS и создает для них в ROS новые контейнеры.
- Mergeout — подметает за нами, когда мы используем DIRECT. Мы не всегда способны грузить столько информации, чтобы получались большие ROS-контейнеры. Поэтому он периодически объединяет небольшие контейнеры ROS в более крупные, очищает данные с пометкой на удаление, работая при этом в фоновом режиме (по времени, заданному в конфигурации).